

OSN Tut 3

XV6 memory mapping

Global memory mapping

XV6 runs on Sv39 (can use Sv49) RISC-V

In RISC-V 64 bit arch, (<https://www.kernel.org/doc/html/latest/riscv/vm-layout.html>) you must have bits 63 - 38 all equal to bit 37 or a page fault will be triggered

So we have the memory mapping as 256 GB block of Userspace, then

Huge gap of addr 0000004000000000 to fffffffbffffffff

and then kernel space where we have fixmap (6MB) (early scratch area for kernel), PCI io (16 MB), vmemmap (4 GB)

(array of `struct page` structures), vmalloc and ioremap (64 GB) (dedicated pool of addresses to give out during allocation), direct mapping of physical mem (124 GB), kasan (32GB) (kernel's address sanitizer), kernel modules (2 GB), kernel (2GB)

RISC-V Linux Kernel SV39

Start addr	Offset	End addr	Size	VM area description
0000000000000000	0	0000003fffffffff	256 GB	user-space virtual memory, different per mm
0000004000000000	+256 GB	fffffffbffffffff	~16M TB	... huge, almost 64 bits wide hole of non-canonical virtual memory addresses up to the -256 GB starting offset of kernel mappings.
				Kernel-space virtual memory, shared between all processes:
ffffffc6fea00000	-228 GB	ffffffc6feffffff	6 MB	fixmap
ffffffc6ff000000	-228 GB	ffffffc6ffffffff	16 MB	PCI io
ffffffc700000000	-228 GB	ffffffc7ffffffff	4 GB	vmemmap
ffffffc800000000	-224 GB	ffffffd7ffffffff	64 GB	vmalloc/ioremap space
ffffffd800000000	-160 GB	fffffffeffffffff	124 GB	direct mapping of all physical memory
ffffffe700000000	-36 GB	fffffffeffffffff	32 GB	kasan
fffffff000000000	-4 GB	fffffff7ffffffff	2 GB	modules, BPF
fffffff800000000	-2 GB	ffffffffffffff	2 GB	kernel

Use the link to see a better representation

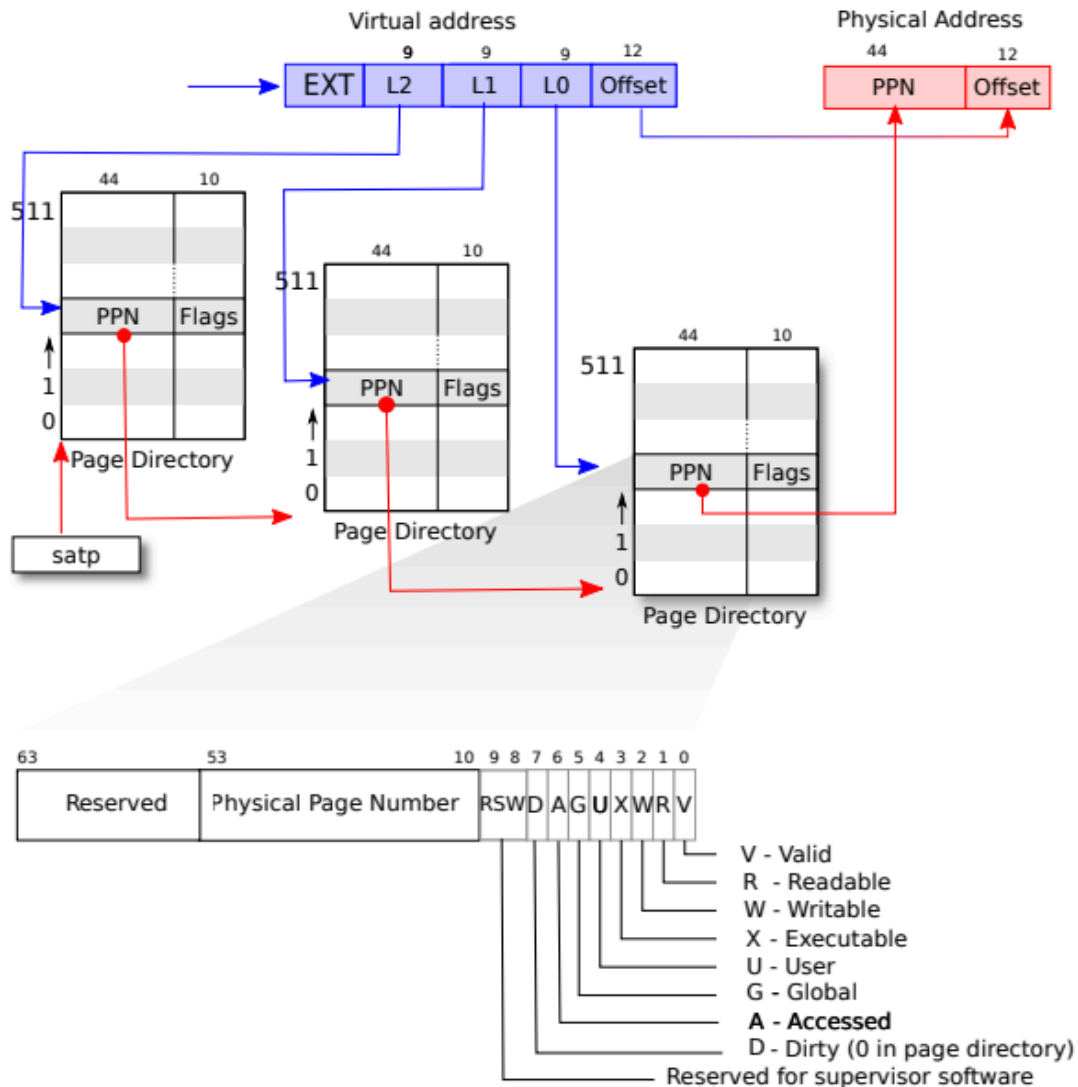


Figure 3.2: RISC-V address translation details.

satp stands for **Supervisor Address Translation and Protection** register - it's the RISC-V CPU register that controls virtual memory translation. It's the RISC-V equivalent of x86's **CR3** register.

This tells where root page table is, then walk down using each PPN for the next level's

table

There's a TLB available.

The kernel stack pages. Each process has its own kernel stack, which is mapped high so that below it xv6 can leave an unmapped guard page. The guard page's PTE is invalid (i.e., PTE_V is not set), so that if the kernel overflows a kernel stack, it will likely cause an exception and the kernel will panic,

(Can the kernel use direct mapped addresses since it is technically available, why does it not?)

User space memory mapping

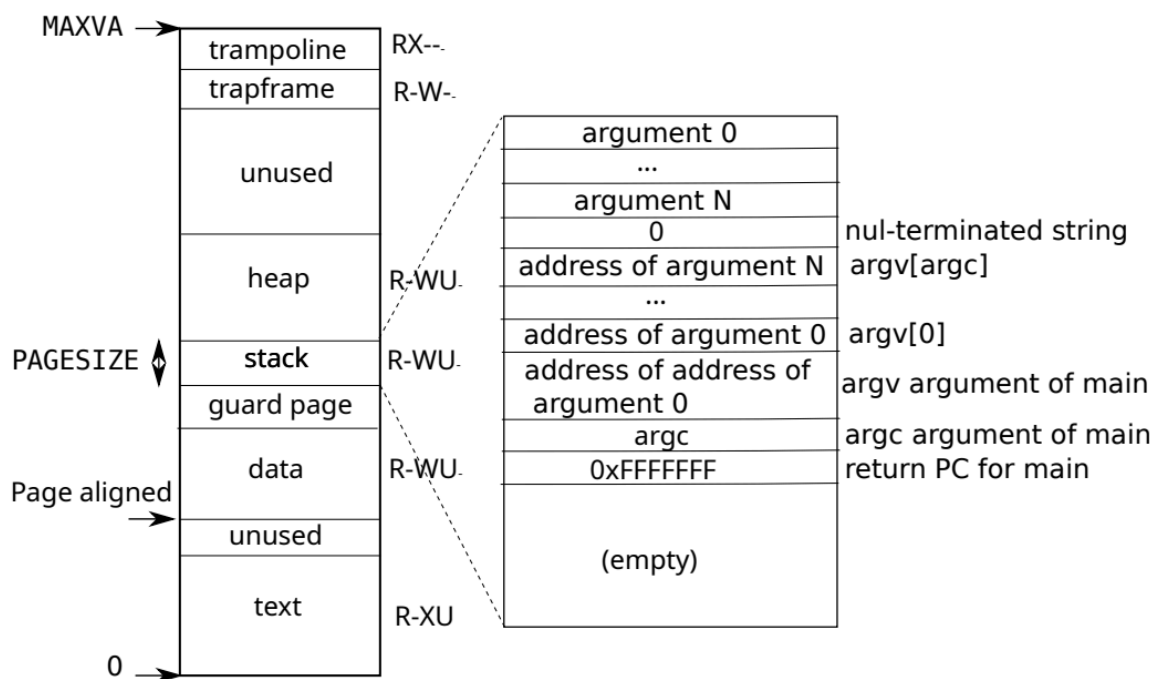


Figure 3.4: A process's user address space, with its initial stack.

Trampoline: Process needs a fixed allocated area which can be used by the kernel after a context switch within the same page table, because hardware doesn't do that, trampoline is mapped at the same location for both user space and kernel space doing this (note: trampoline is a shared entity across all programs and kernel) and this way

kernel has way of running the code in trampoline which then switches the program counter to where the code to handle the context switch is.

Trapframe: Store program's data before going to kernel mode and restore when back (dedicated per process)

XV6's stack is fixed to 1 page only

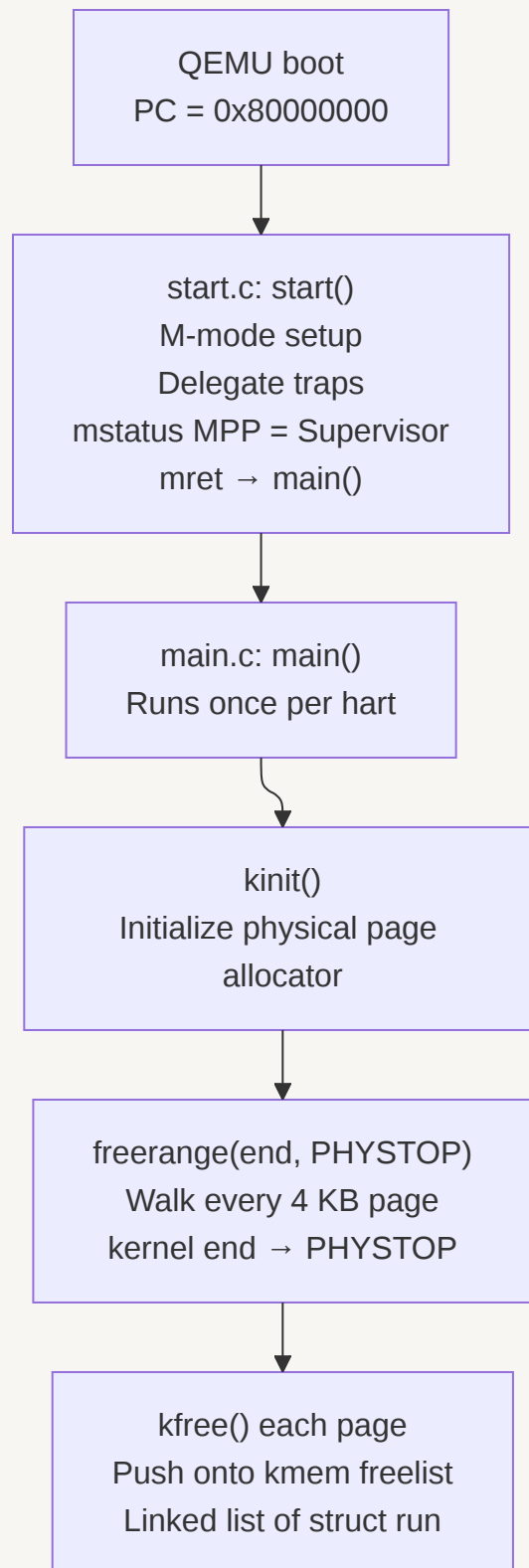
It grows downward

Guard prevents it from overflowing into data

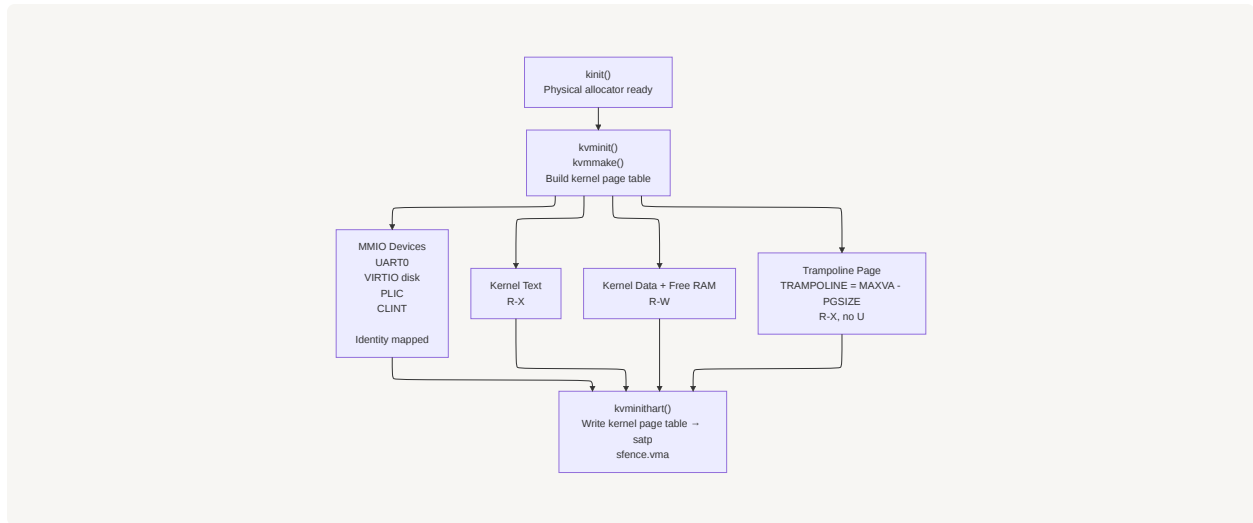
Heap is not limited to 1 page but rather by the physical ram available (PHYSTOP) and MAX_VA (both can be defined at compile time using makefile) Heap grows up

Data is program's data

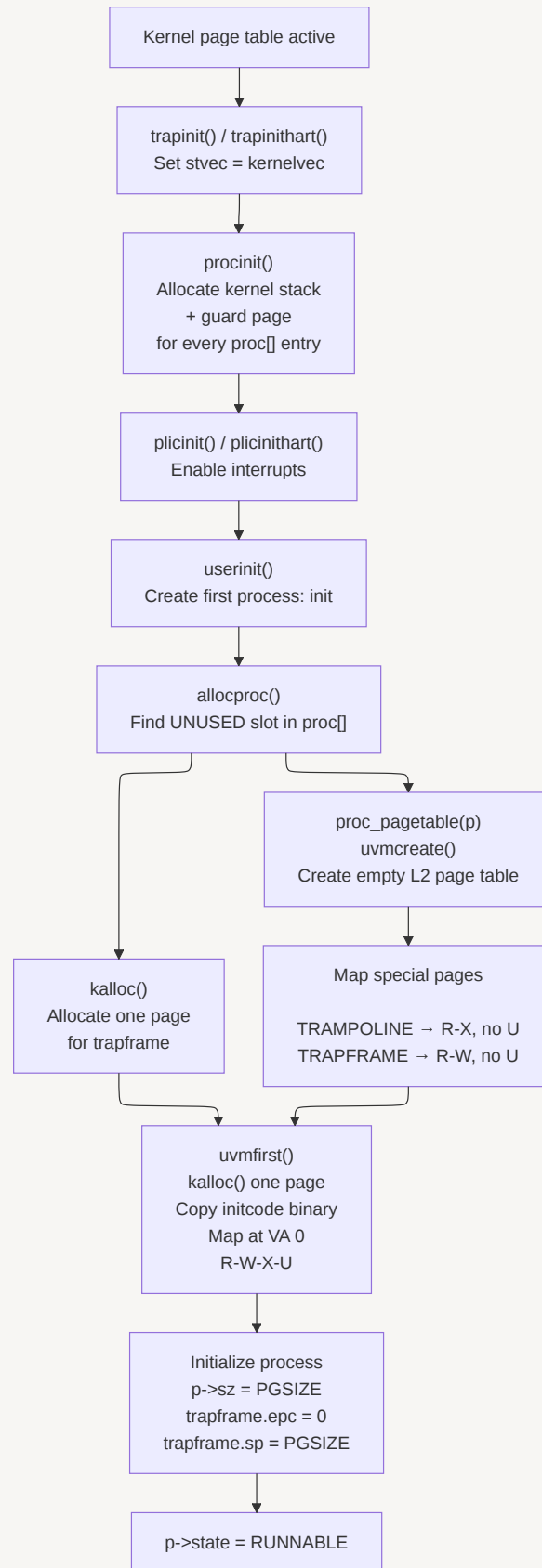
and text is program's code



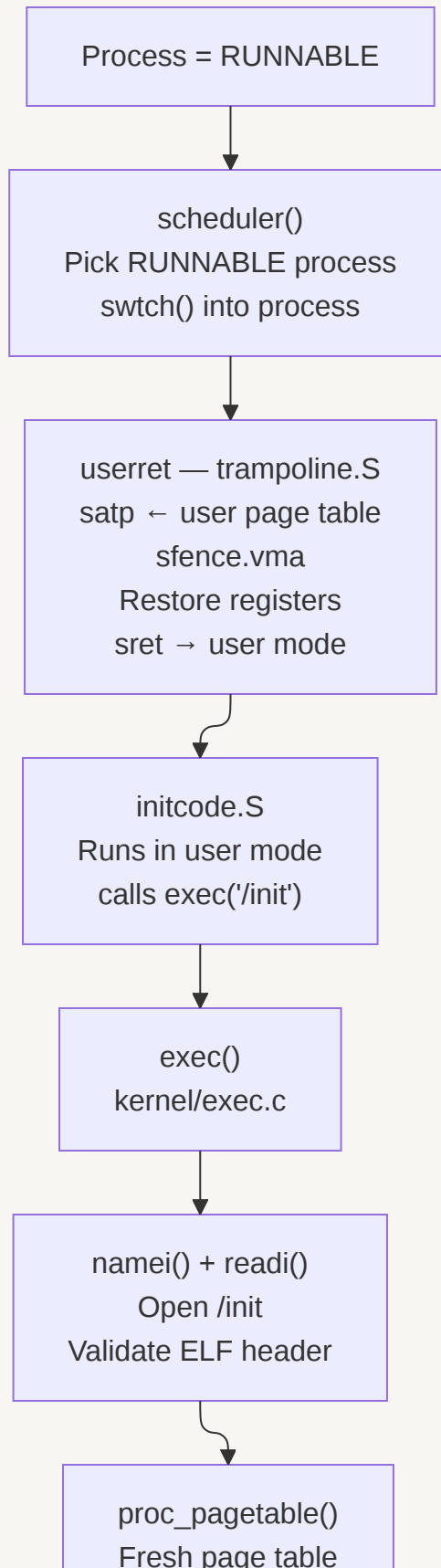
First, the kernel boots in machine mode, switches to supervisor mode, and initializes the physical page allocator by putting all available physical pages onto the free list.

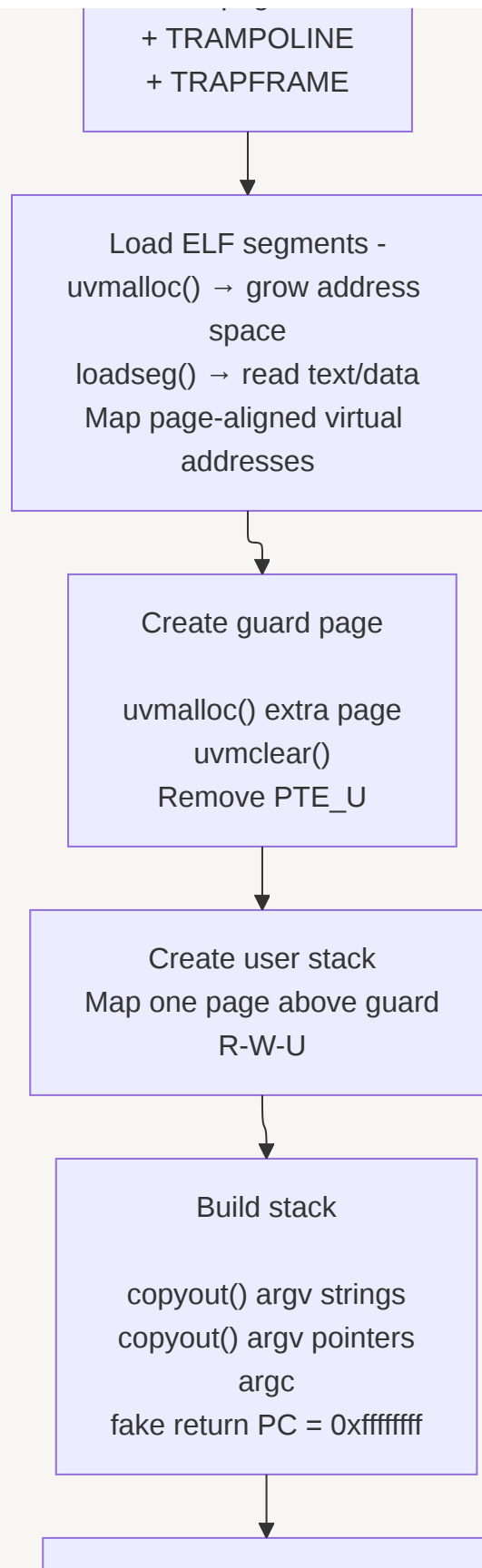


The kernel then constructs its virtual address space: devices are identity mapped, kernel text is executable/read-only, kernel data is writable, and the trampoline is mapped at the highest virtual address

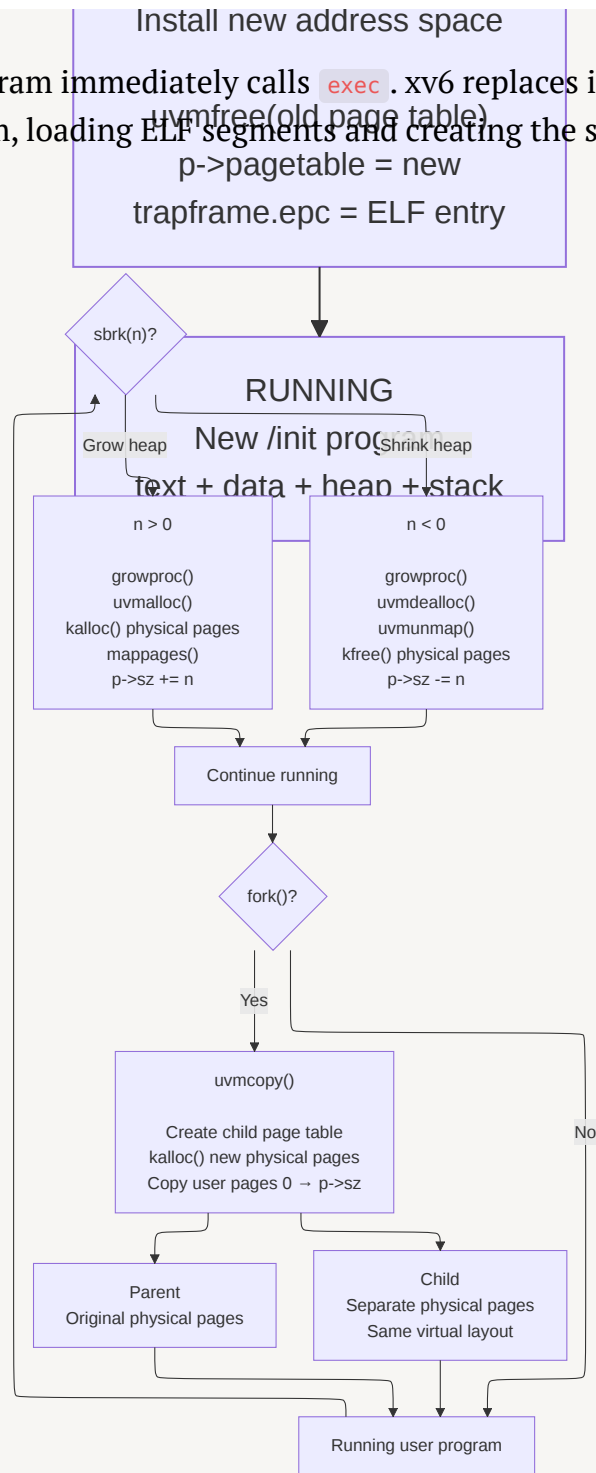


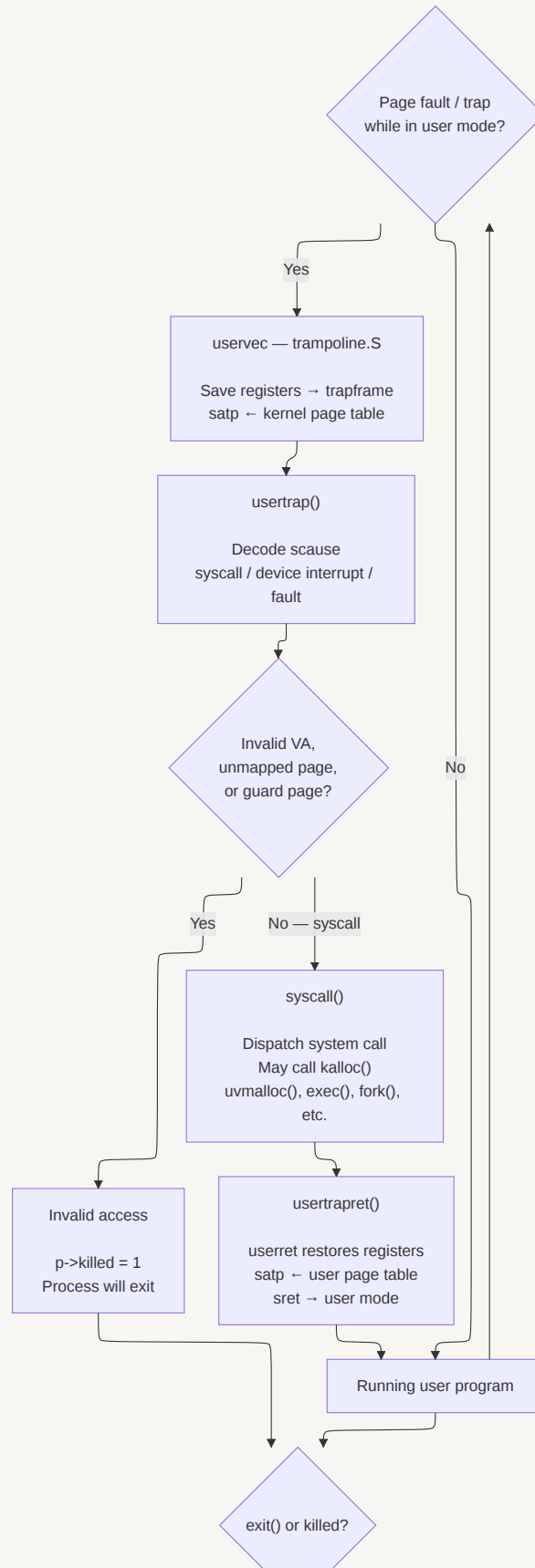
After kernel initialization, xv6 creates the first user process. It gets a trapframe, its own page table, the trampoline/trapframe mappings, and one page containing the initcode

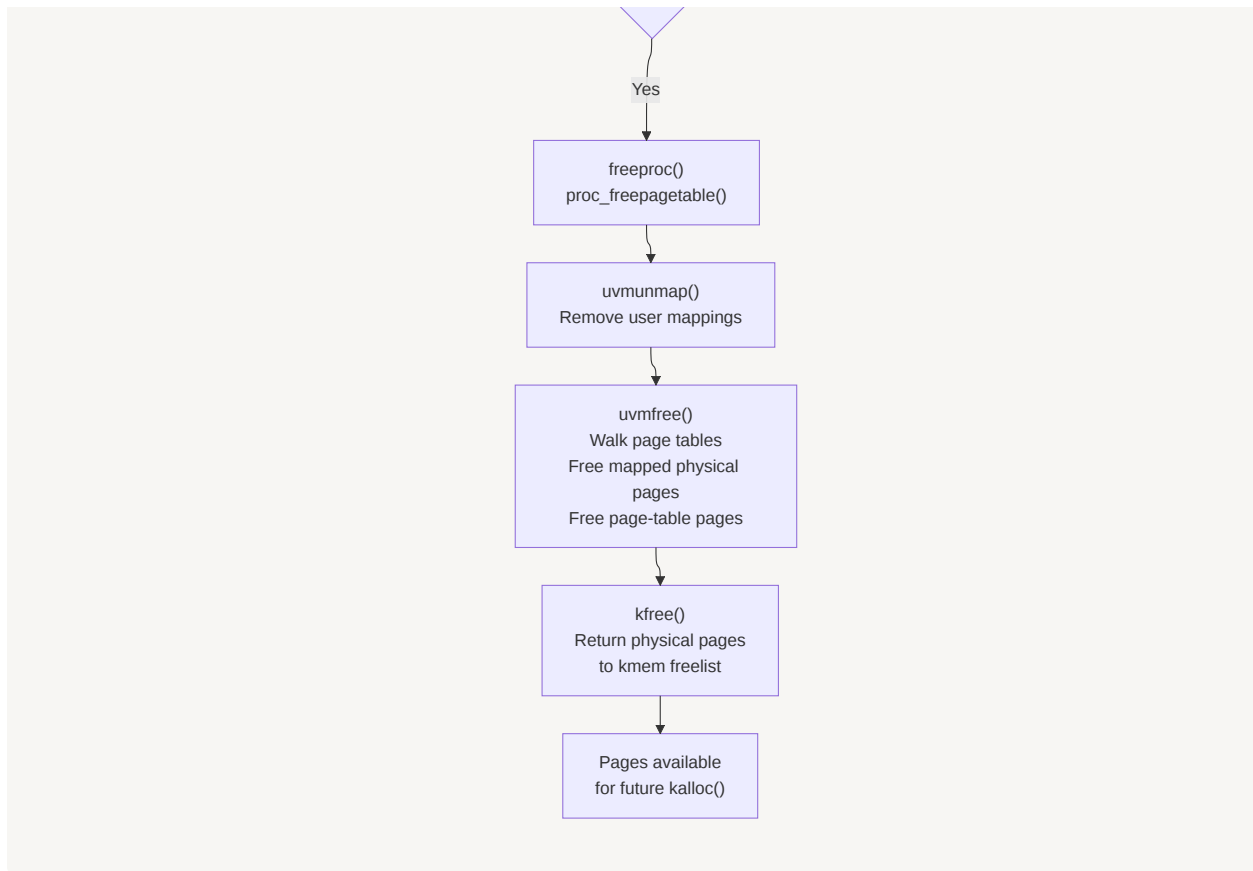




The tiny initcode program immediately calls `exec`. xv6 replaces its address space with the real `/init` program, loading ELF segments and creating the stack and guard page.







Finally, traps switch execution back into the kernel. Invalid accesses kill the process, while normal syscalls return to user mode. When a process exits, xv6 walks its page tables, frees all physical pages, and returns them to the allocator